

# Algoritmos de optimización - Seminario

Nombre y Apellidos: Alejandro Napoleon Thoux Duran

Url:

[https://github.com/catilo/algoritmos\\_optimizacion/blob/main/proyecto/Seminario\\_Algoritmo](https://github.com/catilo/algoritmos_optimizacion/blob/main/proyecto/Seminario_Algoritmo)

Bibliografía utilizada:

- A Common-Sense Guide to Data Structures and Algorithms - Jay Wengrow - April 2023
- Técnica de Diseño de Algoritmos - Guerequeta y Vallecillo - 1998

## Problema:

Combinar cifras y operaciones

Descripción del problema:

- El problema consiste en analizar el siguiente problema y diseñar un algoritmo que lo resuelva. • Disponemos de las 9 cifras del 1 al 9 (excluimos el cero) y de los 4 signos básicos de las operaciones fundamentales: suma(+), resta(-), multiplicación(\*) y división(/)
- Debemos combinarlos alternativamente sin repetir ninguno de ellos para obtener una cantidad dada.

Un ejemplo sería para obtener el 4:  $4+2-6/3*1 = 4$

- Debe analizarse el problema para encontrar todos los valores enteros posibles planteando las siguientes cuestiones:
- ¿Qué valor máximo y mínimo se pueden obtener según las condiciones del problema?
- ¿Es posible encontrar todos los valores enteros posibles entre dicho mínimo y máximo ?



## Respuestas

### ¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

Consideramos que el problema se plantea como una combinatoria de nueve elementos en un string de caracteres, y que no tenemos restricciones a la hora de realizar todas las combinaciones posibles repetición, es posible aplicar la regla del producto para determinar todas las combinaciones posibles.

Cada posición es independiente y puede tomar cualquier valor, por ello la cantidad de posibles combinaciones resulta en:

- Cada una de las 5 posiciones de los numeros, existen 9 opciones

$$9^5 = 59,049$$

- Cada una de las 5 posiciones de las operaciones, existen 4 opciones

$$9^4 = 256$$

El total de combinaciones posibles es el resultado de multiplicar los resultados, resultando en 11,116,544 combinaciones.

### ¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones.

Consideramos la restricción de que los números en el string de la expresión no pueden repetirse.

Se selecciona un grupo de 5 números del conjunto de números enteros del 1 al 9, sin repetición. La cantidad de combinaciones posibles resulta en:

$$\binom{9}{5} = \frac{9!}{5!(9-5)!} = \frac{9!}{5! \cdot 4!} = 126$$

Considerando que por cada combinacion de 5 numeros se realiza una permutacion de los cinco numeros mas una permutacion de las 4 operaciones, se tendria lo siguiente:

- Permutaciones de los numeros

$$P(5) = 5! = 120$$

- Permutaciones de las operaciones

$$P(4) = 4! = 24$$

El total de las expresiones por las permutaciones resulta en  $120 \times 24 = 2.880$  expresiones distintas.

El total de combinaciones posibles es el resultado de multiplicar los resultados de la combinatoria y la permutaciones, resultando en  $126 \times 2.880 = 362.880$  combinaciones.

### Posibilidades sin tener en cuenta todas las restricciones

```
In [16]: from itertools import product, combinations_with_replacement

numeros = range(1, 10)
operadores = ['+', '-', '*', '/']

# Lista para almacenar permutaciones completas
count = 0

# Paso 1: combinar 5 números únicos con los 4 operadores
for seleccion_numeros in product(numeros, repeat=5):
    for perm in product(operadores, repeat=4):
        count += 1
```

```
print(f"Total de permutaciones generadas: {count}")
```

Total de permutaciones generadas: 15116544

## Posibilidades teniendo en cuenta todas las restricciones

```
In [ ]: # Posible combinaciones generadas con la restricciòn
# de que los números en el string no pueden repetirse.

from itertools import permutations, combinations

numeros = range(1, 10)
operadores = ['+', '-', '*', '/']

# Lista para almacenar permutaciones completas
permutaciones_completas = []

# Paso 1: combinar 5 números únicos con los 4 operadores
for seleccion_numeros in combinations(numeros, 5):
    elementos = list(seleccion_numeros)
    for perm in permutations(elementos):
        for oper in permutations(operadores):
            permutaciones_completas.append(perm)

print(f"Total de permutaciones generadas: {len(permutaciones_completas)}")
```

Total de permutaciones generadas: 362880

In [ ]:

## Modelo para el espacio de soluciones

¿Cuál es la estructura de datos que mejor se adapta al problema? Argumentalo. (Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

Respuesta

La primera y segunda versión del algoritmo propuesto, `algoritmo_optimizado()` y `algoritmo_optimizado_v2()` respectivamente, no incluyeron una estructura de datos especiales para la manipulación de datos en el tratamiento del problema. La segunda versión del problema introdujo la función de evaluación `es_candidato_v2()`, la cual implementa la evaluación de combinaciones como candidatos factibles de producir resultados enteros (unas de las restricciones del problema) para ser evaluados. Esta evaluación sigue una regla general de que una operación de división puede generar un resultado fraccional en el caso de que una operación de multiplicación inmediatamente después no cancele o limpie el denominador.

Un análisis más profundo de las posibles combinaciones que se generan partiendo de un número de 5 cifras y las 4 operaciones planteadas, indica que es posible realizar una optimización adicional si se considera que no es necesario evaluar todas las combinaciones de operaciones sobre el mismo número, ya que las 24 combinaciones pueden agruparse en 10 grupos, de los cuales solo es necesario evaluar una combinación

para determinar si todas las demas combinaciones del grupo son candidatas, es decir, si generan un numero entero al ser evaluadas.

Los grupos de 10 combinaciones se plantean de la siguiente manera en la tercera version del algoritmo, algoritmo\_optimizado\_v3():

- ops\_g0 = [['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/']]
- ops\_g1 = [['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/']]
- ops\_g2 = [['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/']]
- ops\_g3 = [['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/']]
- ops\_g4 = [['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/']]
- ops\_g5 = [['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/']]
- ops\_g6 = [['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/']]
- ops\_g7 = [['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/']]
- ops\_g8 = [['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/']]
- ops\_g9 = [['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/'], ['+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/', '+', '-', '/']]

Para cada uno de los grupos definidos (ops\_g0 al ops\_g9) el resultado de la expresion, entero o decimal, es el mismo independientemente de los valores numericos empleados en la combinacion. La version final del algoritmo propuesto utiliza una lista de python para concatenar los grupos en orden desde el grupo ops\_g0 al grupo ops\_g9, de esta manera la funcion de evluacion verifica el primer elemento de la lista para determinar si todo el grupo debe evaluarse como soluciones posibles de cumplir con la restriccion de evaluarse a un numero entero.

## Según el modelo para el espacio de soluciones

¿Cual es la función objetivo?

Respuesta

Considerando las restricciones y el objetivo de obtener una expresion que de como resultado un numero entero igual al pasado como condicion al algoritmo, la funcion objetivo podria formularse como:

$$f(\text{expresion}) = \text{abs}(\text{expresion} - T) = 0$$

Donde T es el valor objetivo y si ademas:

- Si  $f(\text{expresion}) = 0$ , significa que se ha encontrado una combinacion exacta que da el numero objetivo.
- Para cualquier otro valor de  $f(\text{expresion})$  que no de resultado cero, no se cumple la condicion.

¿Es un problema de maximización o minimización?

Respuesta

El problema puede interpretarse como de minimización, pero como se requiere encontrar el valor exacto de T es un problema de satisfacción de restricciones.

## Diseña un algoritmo para resolver el problema por fuerza bruta

Respuesta

```
In [ ]: import time
MILISEGUNDOS = 1000000

def fuerza_bruta(condition: int, debug= None) -> None:

    # Posible combinaciones generadas con la restricción
    # de que los números en el string no pueden repetirse.

    from itertools import permutations, combinations
    import ast

    numeros = range(1, 10)
    operadores = ['+', '-', '*', '/']

    tick = time.time_ns()

    # Paso 1: combinar 5 números únicos con los 4 operadores
    for seleccion_numeros in combinations(numeros, 5): #L1
        for perm in permutations(seleccion_numeros): #L2

            num_perm = "".join([str(e) for e in list(perm)]) #L3

            for oper_permu in permutations(operadores): #L4
                expression = num_perm[0] + str(oper_permu[0]) + num_perm[1] + str(oper_permu[1]) + num_perm[2] + str(oper_permu[2]) + num_perm[3]

                try: #L6
                    result = eval(expression) #L7
                except Exception as exc: #L8
                    print(f"Generated exception: {exc}, expression: {expression}")
                else: #L10
                    if debug is not None: #L11
                        print(f"Expression: {expression}, result: {result}")
                    if result.is_integer() and result == condition: #L12
                        tock = time.time_ns() #L13
                        print(f"Expression {expression} found to satisfy the condition")
                        print(f"Execution Time: {(tock-tick)/MILISEGUNDOS} milis")
                        return #L16

    if debug is not None:
        print(f"No se encontro una expresion para el valor: {condition}")
```

In [220...] fuerza\_bruta(4)

Expression 1/2\*4-3+5 found to satisfy the condition 4  
Execution Time: 0.4487 milisegundos

## Calcula la complejidad del algoritmo por fuerza bruta

### Respuesta

Se analiza el caso general donde es posible variar la cantidad de numeros  $N$  para la combinatoria  $\binom{N}{k} = \binom{9}{5}$ , las lineas identificadas como L1 y L2 en el codigo producen la combinatoria  $\frac{9!}{(9-5)!}$  y L4 se mantiene constante en  $4!$ .

La complejidad queda expresada como  $O(N^k \cdot 4!)$ . Eliminando las constantes y reemplazando el valor de  $k$ , se obtiene una complejidad Polinomica de grado 5:  $O(N^5)$

## Diseña un algoritmo que mejore la complejidad del algortimo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

### Analisis preliminar de los requerimientos para el algoritmo

El algoritmo debe generar la combinatoria de los 9 elementos que definen las expresiones a evaluar (5 números y 4 operadores) de la manera mas eficiente posible considerando la reduccion del uso de memoria y evitando en lo posible la generacion de expresiones que en base a las restricciones planteadas no sean eficientes para su evaluacion. Se plantea en primera instancia el uso de la tecnica de backtracking con recursion para la generacion y eliminacion con poda temprana de las expresiones que no son candidatas para la evaluacion, asi tambien se considero el uso de arboles de expansion para mantener el estado de una rama de expansion para cada expresion y aplicar poda temprana en las ramas que no son candidatas. A continuacion se plantean las principales consideraciones para el analisis:

- El problema se alinea con el uso de backtracking debido a que se requiere generar todas las posibles soluciones validas y en cada paso se evalua parcialmente la expresion podandose las ramas con una funcion de poda que considere las restricciones.
- La solucion no requiere la evaluacion del costo por lo que no es necesario guardar los nodos de las ramas en estructuras de datos especiales a medida que se generan para evaluar el costo de la expresion.
- En el caso particular de que se requiera evaluar las expresiones que cumplen con el valor resultante esperado y calcular el costo de generacion, por ejemplo, las expresiones que obtienen el resultado esperado pero con los numeros mas pequeños, entonces si se necesitaria guardar los nodos en alguna estructura de datos para la evaluacion del costo.
- Se elije descartar para esta implementacion el uso de recursion para evitar complejizar el algoritmo y tener mas control sobre la ejecucion durante las pruebas. Ademias, para esta primera implementacion resulta menos arriesgado el uso de recursion profunda para evitar volcado del stack durante la ejecucion.
- El uso de recursion puede ser una opcion valida en subsecuentes implementaciones para el caso de querer mejorar la modularidad o generalizar el algoritmo.

## Primera version del algoritmo optimizado

En esta primera version del algoritmo se generan las permutaciones en cuatro bucles con iteradores y se define una expresion para el caso en el que avanzar con la evaluacion de la expresion no cumple con los requerimientos. La expresion que interrumpe continuar con la evaluacion del siguiente nodo y se basa en el requerimiento de que el resultado a alcanzar debe ser un entero, entonces en el caso de que la expresion pueda resultar en una fraccion, se interrumpe la ejecucion y se continua con la creacion de la siguiente expresion. Para evaluar la expresion como entero, se considera que si el resultado de la operacion de division es una valor fraccional, entonces todo el resultado de la expresion sera tambien fraccional y se puede descartar.

```
In [64]: def es_candidata(expresion: str, permu: tuple) -> bool:
        """
        # Se evalua que el resultado de la expresion sea un valor entero, en caso co
        # La variable div_index se utiliza como indice para extraer los numero enter
        # y si la operacion modulo retorna un valor distinto de cero, se procede a d
        """

        result = False
        div_index = {v:k for k,v in dict(enumerate(permu)).items()}["/"]
        if int(expresion[div_index]) % int(expresion[div_index + 1]) == 0:
            result = True

        return result
```

```
In [190... from itertools import permutations, combinations
import time

def algoritmo_optimizado(condition: int, debug=None) -> str:

    MICROSEGUNDOS = 1000
    numeros = range(1, 10)
    operadores = ['+', '-', '*', '/']

    tick = time.time_ns()
    # Paso 1: combinar 5 números únicos con los 4 operadores
    for seleccion_numeros in combinations(numeros, 5):
        for perm in permutations(seleccion_numeros):

            num_perm = "".join([str(e) for e in list(perm)])

            for oper_permu in permutations(operadores):
                if not es_candidata(expresion=num_perm, permu=oper_permu):
                    continue

            #Se genera la expresion para su posterior evaluacion
            expression = num_perm[0] + oper_permu[0] + num_perm[1] + oper_pe

            try:
                result = eval(expression)
            except Exception as exc:
                print(f"Generated exception: {exc}, expression: {expression}")
            else:
```

```

        if debug is not None:
            print(f"Expression: {expression}, result: {result}")
        if result == condition:
            print(f"Expression {expression} found to satisfy the con
            tock = time.time_ns()
            print(f"Execution Time: {(tock-tick)/MICROSEGUNDOS} micr
            return "Completed"

    if debug is not None:
        print(f"No se encontro una expresion para el valor: {condition}")

```

In [166... algoritmo\_optimizado(4)

Expression 1\*4/2-3+5 found to satisfy the condition 4  
Execution Time: 449.8 microsegundos

Out[166... 'Completed'

In [78]: algoritmo\_optimizado(4, True)

Expression: 1+3-4/2\*5, result: -6.0  
Expression: 1+3\*4/2-5, result: 2.0  
Expression: 1-3+4/2\*5, result: 8.0  
Expression: 1-3\*4/2+5, result: 0.0  
Expression: 1\*3+4/2-5, result: 0.0  
Expression: 1\*3-4/2+5, result: 6.0  
Expression: 1+3-5\*4/2, result: -6.0  
Expression: 1+3\*5-4/2, result: 14.0  
Expression: 1-3+5\*4/2, result: 8.0  
Expression: 1-3\*5+4/2, result: -12.0  
Expression: 1\*3+5-4/2, result: 6.0  
Expression: 1\*3-5+4/2, result: 0.0  
Expression: 1+4/2-3\*5, result: -12.0  
Expression: 1+4/2\*3-5, result: 2.0  
Expression: 1-4/2+3\*5, result: 14.0  
Expression: 1-4/2\*3+5, result: 0.0  
Expression: 1\*4/2+3-5, result: 0.0  
Expression: 1\*4/2-3+5, result: 4.0  
Expression 1\*4/2-3+5 found to satisfy the condition 4  
Execution Time: 460.6 microsegundos

Out[78]: 'Completed'

## Segunda version del algoritmo optimizado

La primera version del algoritmo presenta un problema en la definicion de la funcion de poda anticipada `es_candidata()`, ya que se considera que cuando la operacion de division tiene como resultado un numero fraccional, toda la expresion resultara fraccional, sin embargo, esto no es generalmente verdadero.

Al realizar una división como:

$$\frac{a}{b}$$

con

$$(a, b \in \mathbb{Z})$$



, puede que el resultado no sea entero, por ejemplo:

$$\frac{7}{4} = 1.75$$

La única manera de que esa fracción vuelva a ser un número entero es que en la misma rama multiplicativa (es decir, antes de que aparezca un(+) o un (-) que cierre el bloque) aparezca una multiplicación por un número (c) que "limpie" el denominador.

Matemáticamente, esto significa:

$$\frac{a}{b} \times c = \frac{a \cdot c}{b}$$

y el resultado será entero si y solo si:

$$b \mid (a \cdot c)$$

es decir, si el denominador divide exactamente al nuevo numerador, en el caso de que no divida exactamente el resultado continua siendo fraccional, como en el siguiente ejemplo:

$$(7 \div 4) \times 3 = \frac{7}{4} \times 3 = \frac{21}{4}$$

Donde:

$$a = 7, \quad b = 4, \quad c = 3$$

Para que esta regla se cumpla y el resultado pueda ser entero, las operaciones de division y multiplicacion deben estar en el mismo bloque de la expresion o lo que es lo mismo ser consecutivas. Si despues de la division la operacion siguiente es una suma o resta, entonces no es posible compensar el denominador y el resultado permanece como fraccional:

$$(7 \div 4) + 5 = \frac{7}{4} + 5$$

La regla de poda a implementar en la funcion es\_candidata\_v2() se resume de la siguiente manera:

- La división crea un denominador.
- Para que el resultado vuelva a ser entero, ese denominador debe dividir exactamente al producto del numerador por las multiplicaciones posteriores dentro del bloque.
- Si no se cumple, la expresión completa nunca será un entero.

```
In [3]: def es_candidata_v2(nums: list, ops: list) -> bool:
        """
        # Se evalua que el resultado de la expresion sea un valor entero.
        # El parametro de entrada nums espera una lista de los 5 numeros enteros que
        # El parametro de entrada ops espera una lista con las cuatro operaciones a
        # La funcion devuelve False si la expresion se evalua a un valor no entero,
        # la expresion se evalua a un entero.
        """
```

```

current_numer = nums[0]
current_denom = 1
division_in_block = False

for k, op in enumerate(ops):
    if op == '*':
        current_numer *= nums[k+1]
    elif op == '/':
        # única división; el divisor es el siguiente número
        current_denom = nums[k+1]
        division_in_block = True
        # no multiplicamos el divisor con el numerador
    elif op in ['+', '-']:
        # cierre de bloque multiplicativo
        if division_in_block:
            if current_numer % current_denom != 0:
                return False # NO entero
            # reiniciar para el siguiente bloque
            current_numer = nums[k+1]
            current_denom = 1
            division_in_block = False
        else:
            raise ValueError(f"Operador no válido: {op}")

# comprobar el último bloque
if division_in_block:
    if current_numer % current_denom != 0:
        return False #No entero

return True

```

In [177...

```

from itertools import permutations, combinations
import time

def algoritmo_optimizado_v2(condition: int, debug=None) -> None:

    # Posible combinaciones generadas con la restricción
    # de que los números en el string no pueden repetirse.

    MICROSEGUNDOS = 1000
    numeros = range(1, 10)
    operadores = ['+', '-', '*', '/']

    tick = time.time_ns()
    # Paso 1: combinar 5 números únicos con los 4 operadores
    for seleccion_numeros in combinations(numeros, 5):
        for perm in permutations(seleccion_numeros):

            num_perm = "".join([str(e) for e in list(perm)])
            #for comb in combinations(operadores, 4):
            for oper_permu in permutations(operadores):

                if not es_candidata_v2(nums=perm, ops=oper_permu):
                    continue

            #Se genera la expresión para su posterior evaluación
            expression = num_perm[0] + oper_permu[0] + num_perm[1] + oper_pe

```

```

try:
    result = eval(expression)
except Exception as exc:
    print(f"Generated exception: {exc}, expression: {expression}")
else:
    if result == condition:
        tock = time.time_ns()
        print(f"Expression {expression} found to satisfy the con
        print(f"Execution Time: {(tock-tick)/MICROSEGUNDOS} micr
        return
if debug is not None:
    tock = time.time_ns()
    print(f"No se encontro una expresion para el valor: {condition}")
    print(f"Execution Time: {(tock-tick)/MICROSEGUNDOS} microsegundos")

```

In [181... algoritmo\_optimizado\_v2(4)

Expression 1/2\*4-3+5 found to satisfy the condition 4  
Execution Time: 69.5 microsegundos

## Mejora sobre funcion de evaluacion

```

In [1]: from itertools import permutations, combinations
import time

def algoritmo_optimizado_v3(condition: int, debug=None) -> str:

    # Las 24 combinaciones pueden agruparse en 10 grupos.
    ops_g0 = [['*', '+', '-', '/'], ['+', '*', '-', '/'], ['-', '*', '+', '/'], ['*',
    ops_g1 = [['/', '+', '-', '*'], ['/', '+', '*', '-'], ['/', '-', '+', '*'], ['/',
    ops_g2 = [['+', '/', '-', '*'], ['-', '/', '+', '*']]
    ops_g3 = [['*', '+', '/', '-'], ['*', '-', '/', '+']]
    ops_g4 = [['+', '-', '/', '*'], ['-', '+', '/', '*']]
    ops_g5 = [['+', '-', '*', '/'], ['-', '+', '*', '/']]
    ops_g6 = [['+', '*', '/', '-'], ['-', '*', '/', '+']]
    ops_g7 = [['-', '/', '*', '+'], ['+', '/', '*', '-']]
    ops_g8 = [['*', '/', '+', '-'], ['*', '/', '-', '+']]
    ops_g9 = [['/', '*', '+', '-'], ['/', '*', '-', '+']]

    OPEX = ops_g0 + ops_g1 + ops_g2 + ops_g3 + ops_g4 + ops_g5 + ops_g6 + ops_g7
    CONTROL = [(4,4),(4,8),(2,10),(2,12),(2,14),(2,16),(2,18),(2,20),(2,22),(2,24)]

    MICROSEGUNDOS = 1000

    numeros = range(1, 10)
    expression = ""

    tick = time.time_ns()
    # Paso 1: combinar 5 números únicos con los 4 operadores
    for seleccion_numeros in combinations(numeros, 5): #L1
        for perm in permutations(seleccion_numeros): #L2

            LS = 0 #L3
            IDX = 0 #L4
            IDX_INLOOP = 0 #L5
            IN_LOOP = False #L6
            NOT_FOUND = True #L7

```

```

while(LS < 10 and NOT_FOUND): #L8

    if es_candidata_v2(nums=perm, ops=OPEX[IDX]): #L9
        IN_LOOP = True #L10
        IDX_INLOOP = CONTROL[LS][0] #L11

    else: #L12
        IN_LOOP = False #L13
        IDX = CONTROL[LS][1] #L14
        LS += 1

    # Bucle interno para la evaluacion del grupo si
    # este es candidato.

    if IN_LOOP: #L15

        for _ in range(IDX_INLOOP): #L16

            expression = str(perm[0]) + OPEX[IDX][0] + str(perm[1])
            res = eval(expression) #L18

            if res == condition: #L19
                NOT_FOUND = False #L20
                break #L21

            IDX += 1 #L22

        LS +=1 #L23
        IN_LOOP = False #L24

    if not NOT_FOUND: #L25
        tock = time.time_ns()
        print(f"Execution time: {(tock-tick)/MICROSEGUNDOS} microsegundo")
        return expression

if debug is not None:
    print(f"No se encontro una expresion para el valor: {condition}")
    tock = time.time_ns()
    print(f"Execution Time: {(tock-tick)/MICROSEGUNDOS} microsegundos")

```

In [5]: algoritmo\_optimizado\_v3(condition=4)

Execution time: 50.4 microsegundos

Out[5]: '1/2\*4-3+5'

## Complejidad de la ultima version del Algoritmo Optimizado

En la ultima version del algoritmo (algoritmo\_optimizado\_v3) se identifican los diferentes pasos dentro del algoritmo con el identificador Lx, para de esta forma analizar el tiempo de ejecucion de esas expresiones. A continuacion se detalla el analisis:

- L1: Se generan todas las combinaciones de 5 numeros distintos tomados de un conjunto de 9 numeros  $\binom{9}{5} = 126$
- L2: Por cada combinacion de L1 se permuta  $5! = 120$
- L8: El bucle While ejecuta como mucho 10 iteraciones por cada permutacion.

- L16: El bucle for ejecuta como maximo cada una de las evaluaciones para los 10 grupos. Sumando en el peor de los casos para el bucle WHILE y el bucle FOR 24 evaluaciones, lo que es igual a  $4! = 24$
- L17 y L18: Se consideran operaciones constantes.

El resultado final de la cantidad de permutaciones del algoritmo resulta en  $\binom{9}{5} \cdot 5! \cdot 4! = 362.880$ , por lo que la complejidad resulta similar al caso de fuerza bruta, Polinomial de grado 5:  $O(N^5)$ . Sin embargo, esta version del algoritmo reduce el numero de evaluaciones innecesarias implementando la funcion de poda `es_candidata_v2` y la agrupacion de las expresiones. Esta mejora es del tipo constante, no asintotica.

## Evaluacion Comparativa del Tiempo de Ejecucion de las tres versiones del Algoritmo Optimizado para la condicion de entrada igual a 4.

In [184...

```
import numpy as np

execution_v3 = np.zeros(35)

for i in range(35):
    tick = time.time_ns()

    algoritmo_optimizado_v3(condition=4)

    tock = time.time_ns()
    execution_v3[i] = ((tock-tick)/1000)
```

```

Execution time: 51.1 microsegundos
Execution time: 35.4 microsegundos
Execution time: 55.4 microsegundos
Execution time: 27.2 microsegundos
Execution time: 25.2 microsegundos
Execution time: 24.7 microsegundos
Execution time: 24.8 microsegundos
Execution time: 24.9 microsegundos
Execution time: 24.4 microsegundos
Execution time: 23.2 microsegundos
Execution time: 23.8 microsegundos
Execution time: 23.3 microsegundos
Execution time: 23.2 microsegundos
Execution time: 24.4 microsegundos
Execution time: 23.4 microsegundos
Execution time: 24.0 microsegundos
Execution time: 23.9 microsegundos
Execution time: 24.5 microsegundos
Execution time: 24.1 microsegundos
Execution time: 23.5 microsegundos
Execution time: 28.7 microsegundos
Execution time: 25.4 microsegundos
Execution time: 24.1 microsegundos
Execution time: 23.4 microsegundos
Execution time: 23.2 microsegundos
Execution time: 24.2 microsegundos
Execution time: 24.3 microsegundos
Execution time: 25.6 microsegundos
Execution time: 66.4 microsegundos
Execution time: 40.2 microsegundos
Execution time: 26.2 microsegundos
Execution time: 25.9 microsegundos
Execution time: 24.6 microsegundos
Execution time: 234.4 microsegundos
Execution time: 72.5 microsegundos

```

In [134... `print(f"Excution stats, mean: {execution_v3.mean()} microsegundos, max: {executi`

Excution stats, mean: 52.81428571428571 microsegundos, max: 112.8, min: 32.1

In [185...

```

import numpy as np

execution_v2 = np.zeros(35)

for i in range(35):
    tick = time.time_ns()

    algoritmo_optimizado_v2(condition=4)

    tock = time.time_ns()
    execution_v2[i] = ((tock-tick)/1000)

```

Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 67.2 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 49.4 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 43.8 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 43.0 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 46.9 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 39.7 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 40.3 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 40.4 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 40.7 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 128.6 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 41.6 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 39.3 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 39.8 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 39.2 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 40.1 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 39.3 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 41.5 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 41.9 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 66.8 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 68.9 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 67.1 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 73.3 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 67.1 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 85.5 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 67.7 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 63.9 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 63.7 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 62.2 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 62.9 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 62.7 microsegundos

Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 62.9 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 266.6 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 151.2 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 61.2 microsegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 56.2 microsegundos

In [186... `print(f"Excution stats, mean: {execution_v2.mean()} microsegundos, max: {executi`

Excution stats, mean: 85.02571428571429 microsegundos, max: 347.1, min: 50.7

In [169... `import numpy as np`  
  
`execution_v2 = np.zeros(35)`  
  
`for i in range(35):`  
    `tick = time.time_ns()`  
  
    `algoritmo_optimizado(condition=4)`  
  
    `tock = time.time_ns()`  
    `execution_v2[i] = ((tock-tick)/1000)`



Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 437.8 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 297.2 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 287.2 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 291.9 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 276.9 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 286.2 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 369.7 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 292.4 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 445.3 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 295.0 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 676.9 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 600.9 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 350.6 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 374.1 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 349.7 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 346.9 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 300.4 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 293.2 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 287.2 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 286.7 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 290.3 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 281.3 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 338.2 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 471.3 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 443.9 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 325.8 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 284.6 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 291.8 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 282.3 microsegundos  
Expression  $1^4/2-3+5$  found to satisfy the condition 4  
Execution Time: 285.5 microsegundos

Expression  $1*4/2-3+5$  found to satisfy the condition 4  
 Execution Time: 282.1 microsegundos  
 Expression  $1*4/2-3+5$  found to satisfy the condition 4  
 Execution Time: 280.3 microsegundos  
 Expression  $1*4/2-3+5$  found to satisfy the condition 4  
 Execution Time: 273.2 microsegundos  
 Expression  $1*4/2-3+5$  found to satisfy the condition 4  
 Execution Time: 280.6 microsegundos  
 Expression  $1*4/2-3+5$  found to satisfy the condition 4  
 Execution Time: 287.2 microsegundos

In [170... `print(f"Excution stats, mean: {execution_v2.mean()} microsegundos, max: {executi`

Excution stats, mean: 346.0685714285715 microsegundos, max: 700.9, min: 278.0

In [196...

```
import numpy as np

fuerza_bruta_v = np.zeros(35)

for i in range(35):
    tick = time.time_ns()

    fuerza_bruta(4)

    tock = time.time_ns()
    fuerza_bruta_v[i] = ((tock-tick)/1000)
```

Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.8588 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.5307 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.5164 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.7921 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.7084 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4318 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4362 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4329 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4458 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4405 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4353 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4449 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4341 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4326 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.9983 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.6019 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 1.4922 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 1.0608 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.7808 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 1.2459 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.6153 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.5807 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4558 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4316 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4207 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4334 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4306 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4153 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4178 milisegundos  
Expression  $1/2*4-3+5$  found to satisfy the condition 4  
Execution Time: 0.4201 milisegundos

Expression  $1/2*4-3+5$  found to satisfy the condition 4  
 Execution Time: 0.4192 milisegundos  
 Expression  $1/2*4-3+5$  found to satisfy the condition 4  
 Execution Time: 0.4256 milisegundos  
 Expression  $1/2*4-3+5$  found to satisfy the condition 4  
 Execution Time: 0.4151 milisegundos  
 Expression  $1/2*4-3+5$  found to satisfy the condition 4  
 Execution Time: 0.4159 milisegundos  
 Expression  $1/2*4-3+5$  found to satisfy the condition 4  
 Execution Time: 0.4181 milisegundos

In [197... `print(f"Excution stats, mean: {fuerza_bruta_v.mean()} microsegundos, max: {fuerz`

Excution stats, mean: 589.5914285714287 microsegundos, max: 1532.8, min: 421.2

**Resultados del Analisis de Tiempos de Ejecucion por Algoritmo para condicion intermedia igual a 4.**

| Algoritmo               | Tiempo Medio | Tiempo Maximo | Tiempo Minimo |
|-------------------------|--------------|---------------|---------------|
| fuerza_bruta            | 589.6 us     | 1532.8 us     | 421.2 us      |
| algoritmo_optimizado    | 346.06 us    | 700.9 us      | 278.0 us      |
| algoritmo_optimizado_v2 | 85.02 us     | 347.1 us      | 50.7 us       |
| algoritmo_optimizado_v3 | 52.81 us     | 112.8 us      | 32.1 us       |

A partir de la tabla comparativa para la evaluacion con la condicion de entrada igual a 4, se puede apreciar que a medida que se avanza con la evolucion del algoritmo a version mas optimizadas el tiempo medio de ejecucion, y tambien los maximo y minimos, denotan una mejora paulatina. La mayor diferencia puede apreciarse entre el tiempo medio de ejecucion del algoritmo de fuerza bruta y la tercera version del algoritmo optimizado, donde se observa una mejora de alrededor de 11 veces o un orden de magnitud. Es importante tener en cuenta la primera version del algoritmo (algoritmo\_optimizado) no es un candidato valido ya que la funcion de evaluacion utilizada en la poda no es la correcta, sin embargo, se mantiene en la tabla como referencia.

## Evaluacion comparativa del Tiempo de Ejecucion maximo de las tres versiones del Algoritmo Optimizado.

In [206... `import numpy as np`

```

execution_v3 = np.zeros(35)

for i in range(35):
    tick = time.time_ns()

    algoritmo_optimizado_v3(condition=1000)

    tock = time.time_ns()
    execution_v3[i] = ((tock-tick)/1000)

```

In [214... `print(f"Excution stats, mean: {execution_v3.mean()/1000} milisegundos, max: {exe`

Excution stats, mean: 670.8898142857144 milisegundos, max: 743.0924, min: 648.2565

```
In [208... import numpy as np

execution_v2 = np.zeros(35)

for i in range(35):
    tick = time.time_ns()

    algoritmo_optimizado_v2(condition=1000)

    tock = time.time_ns()
    execution_v2[i] = ((tock-tick)/1000)
```

```
In [209... print(f"Excution stats, mean: {execution_v2.mean()/1000} milisegundos, max: {exe

Excution stats, mean: 691.7822114285714 milisegundos, max: 798.7138, min: 659.1211
```

```
In [210... import numpy as np

execution = np.zeros(35)

for i in range(35):
    tick = time.time_ns()

    algoritmo_optimizado(condition=1000)

    tock = time.time_ns()
    execution[i] = ((tock-tick)/1000)
```

```
In [217... print(f"Excution stats, mean: {execution.mean()/1000} milisegundos, max: {execut

Excution stats, mean: 682.4748114285715 milisegundos, max: 708.8605, min: 671.1725
```

```
In [212... import numpy as np

fuerza_bruta_v = np.zeros(35)

for i in range(35):
    tick = time.time_ns()

    fuerza_bruta(1000)

    tock = time.time_ns()
    fuerza_bruta_v[i] = ((tock-tick)/1000)
```

```
In [215... print(f"Excution stats, mean: {fuerza_bruta_v.mean()/1000} milisegundos, max: {f

Excution stats, mean: 3120.2849200000005 milisegundos, max: 5020.6358, min: 2266.9788
```

Resultados del Analisis de Tiempos de Ejecucion por Algoritmo para el peor caso donde no se encuentra la condicion.

| Algoritmo    | Tiempo Medio | Tiempo Maximo | Tiempo Minimo |
|--------------|--------------|---------------|---------------|
| fuerza_bruta | 3120 ms      | 5020 ms       | 2267 ms       |

| Algoritmo               | Tiempo Medio | Tiempo Maximo | Tiempo Minimo |
|-------------------------|--------------|---------------|---------------|
| algoritmo_optimizado    | 682 ms       | 709 ms        | 671 ms        |
| algoritmo_optimizado_v2 | 692 ms       | 799 ms        | 659ms         |
| algoritmo_optimizado_v3 | 670 ms       | 743 ms        | 648 ms        |

A partir de la tabla comparativa para la evaluacion de los tiempos de ejecucion maximos, se puede apreciar que a medida que se avanza con la evolucion del algoritmo a version mas optimizadas el tiempo medio de ejecucion, y tambien los maximo y minimos, denotan una mejora paulatina. La mayor diferencia puede apreciarse entre el tiempo medio de ejecucion del algoritmo de fuerza bruta y la tercera version del algoritmo optimizado, donde se observa una mejora de alrededor de 4,65 veces. Es importante tener en cuenta la primera version del algoritmo (algoritmo\_optimizado) no es un candidato valido ya que la funcion de evaluacion utilizada en la poda no es la correcta, sin embargo, se mantiene en la tabla como referencia.

**Debe analizarse el problema para encontrar todos los valores enteros posibles planteando las siguientes cuestiones:**

**¿Qué valor máximo y mínimo se pueden obtener según las condiciones del problema?**

In [143...

```
import time
from itertools import permutations, combinations

def algoritmo_optimizado_v3(condition: int, debug=None) -> tuple:

    # Lista para almacenar permutaciones completas
    Max = 0
    Min = 0
    register = {}

    ops_g0 = [['*', '+', '-', '/'], ['+', '*', '-', '/'], ['-', '*', '+', '/'], ['*',
ops_g1 = [['/', '+', '-', '*'], ['/', '+', '*', '-'], ['/', '-', '+', '*'], ['/',
ops_g2 = [['+', '/', '-', '*'], ['-', '/', '+', '*']]
ops_g3 = [['*', '+', '/', '-'], ['*', '-', '/', '+']]
ops_g4 = [['+', '-', '/', '*'], ['-', '+', '/', '*']]
ops_g5 = [['+', '-', '*', '/'], ['-', '+', '*', '/']]
ops_g6 = [['+', '*', '/', '-'], ['-', '*', '/', '+']]
ops_g7 = [['-', '/', '*', '+'], ['+', '/', '*', '-']]
ops_g8 = [['*', '/', '+', '-'], ['*', '/', '-', '+']]
ops_g9 = [['/', '*', '+', '-'], ['/', '*', '-', '+']]

    OPEX = ops_g0 + ops_g1 + ops_g2 + ops_g3 + ops_g4 + ops_g5 + ops_g6 + ops_g7
    CONTROL = [(4,4),(4,8),(2,10),(2,12),(2,14),(2,16),(2,18),(2,20),(2,22),(2,2

    numeros = range(1, 10)
    expression = ""

    tick = time.time_ns()
```

```

# Paso 1: combinar 5 números únicos con los 4 operadores
for seleccion_numeros in combinations(numeros, 5):
    for perm in permutations(seleccion_numeros):

        LS = 0
        IDX = 0
        IDX_INLOOP = 0
        IN_LOOP = False
        NOT_FOUND = True

        while(LS < 10 and NOT_FOUND):

            if es_candidata_v2(nums=perm, ops=OPEX[IDX]):
                IN_LOOP = True
                IDX_INLOOP = CONTROL[LS][0]

            else:
                IN_LOOP = False
                IDX = CONTROL[LS][1]
                LS += 1

            # Bucle interno para la evaluacion del grupo si
            # este es candidato.

            if IN_LOOP:

                for _ in range(IDX_INLOOP):

                    expression = str(perm[0]) + OPEX[IDX][0] + str(perm[1])
                    res = eval(expression)

                    #Guardan los valores obtenidos, max y min
                    if res > Max:
                        Max = res
                    elif res < Min:
                        Min = res

                    if not register.get(res, False):
                        register[res] = True

                    if res == condition:
                        NOT_FOUND = False
                        break

                    IDX += 1

                LS +=1
                IN_LOOP = False

            if not NOT_FOUND:
                tock = time.time_ns()
                print(f"Execution time: {(tock-tick)/1000} microsegundos")
                return expression

if debug is not None:
    print(f"Max: {Max} and Min: {Min}")
    print(f"Registro de números: {register}")
    return (int(Min),int(Max),register)

```

In [144... `min, max, valores = algoritmo_optimizado_v3(condition=1000, debug=True)`

Max: 77.0 and Min: -69.0

Registro de números: {0.0: True, 4.0: True, 2.0: True, 6.0: True, -6.0: True, 8.0: True, 14.0: True, -12.0: True, -15.0: True, 9.0: True, 19.0: True, -5.0: True, 5.0: True, 7.0: True, 13.0: True, -9.0: True, 10.0: True, 11.0: True, 21.0: True, -3.0: True, 12.0: True, 15.0: True, 17.0: True, 16.0: True, 3.0: True, 1.0: True, -2.0: True, -1.0: True, -4.0: True, -8.0: True, -19.0: True, 23.0: True, 25.0: True, 20.0: True, -10.0: True, -18.0: True, -23.0: True, 27.0: True, 29.0: True, -7.0: True, 22.0: True, -21.0: True, -27.0: True, 31.0: True, 33.0: True, 18.0: True, 26.0: True, -14.0: True, -24.0: True, -31.0: True, 35.0: True, 37.0: True, -11.0: True, 28.0: True, -25.0: True, -30.0: True, 34.0: True, 36.0: True, 24.0: True, -16.0: True, -35.0: True, 39.0: True, -17.0: True, 41.0: True, -40.0: True, 44.0: True, -20.0: True, 46.0: True, 30.0: True, -37.0: True, -13.0: True, 43.0: True, -43.0: True, 47.0: True, 49.0: True, -49.0: True, 53.0: True, 55.0: True, -51.0: True, 57.0: True, -58.0: True, 62.0: True, 64.0: True, 32.0: True, -32.0: True, -22.0: True, -67.0: True, 71.0: True, 73.0: True, 38.0: True, -29.0: True, -34.0: True, 42.0: True, -42.0: True, -39.0: True, -36.0: True, 40.0: True, -45.0: True, -28.0: True, 50.0: True, -48.0: True, 52.0: True, 56.0: True, -53.0: True, -50.0: True, 54.0: True, 58.0: True, -60.0: True, -57.0: True, 61.0: True, 65.0: True, -69.0: True, -66.0: True, 70.0: True, -26.0: True, 74.0: True, 45.0: True, -41.0: True, 51.0: True, -47.0: True, 59.0: True, -56.0: True, 60.0: True, 66.0: True, -65.0: True, 69.0: True, 75.0: True, 48.0: True, -52.0: True, -59.0: True, -55.0: True, 67.0: True, -68.0: True, -64.0: True, 68.0: True, -44.0: True, 76.0: True, -63.0: True, 77.0: True, 63.0: True, -33.0: True, -38.0: True, -46.0: True, -54.0: True, -62.0: True, -61.0: True, 72.0: True}

**Respuesta:** Los valores maximo y minimo que se pueden obtener a partir de las condiciones son:

- Maximo: 77
- Minimo: -69

**¿Es posible encontrar todos los valores enteros posibles entre dicho mínimo y máximo ?**

In [151... `for idx in range(min,max+1):  
 if not valores.get(idx, False):  
 print(f"Numero {idx} no esta presente!")`

**Respuesta:** Si, es posible encontrar todos los valores enteros entre los valores maximo y minimo.

**Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo**

Bibliografia utilizada:

- A Common-Sense Guide to Data Structures and Algorithms - Jay Wengrow - April 2023
- Tecnica de Diseño de Algoritmos - Guerequeta y Vallecillo - 1998
- Big O Notation Simplified - Michael Brown - 2024

**Describe brevemente las lineas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta**



## incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Debido a la naturaleza combinatoria del problema un incremento en la cantidad de numeros  $K$  ( $K = 5$  para este problema) en la operacion de combinacion, podrian aumentar la actual complejidad polinomica de grado 5 del algoritmo de manera considerable. Como opciones de estudio para el futuro y posibles mejoras del algoritmo planteo las siguientes:

- Profundizar en una funcion de poda que permita eliminar expresiones que puedan dar resultados duplicados al ser evaluadas, de esta manera evitando la doble evaluacion. Por ejemplo identificando las operaciones conmutativas con los mismos numero  $a+b = b+a$
- Utilizar rangos minimos y maximos posibles para podar la rama si el valor de la expresion se aleja mucho de la condicion.
- Analizar el uso de una Heuristica para priorizar las operaciones, por ejemplo si el valor de la condicion es pequeño, priorizar expresiones con divisiones de numeros grandes y si la condicion es un numero grande priorizar las expresiones con multiplicaciones por numeros grandes.